This style of hook is easy to read, but it does split the setup and teardown logic into two halves that we have to keep track of. If you want to keep all this related code together, you can use an around hook instead.

Favor before over after for Data Cleanup



When your database cleanup logic doesn't fit neatly into a transactional around hook, you'll want to consider either a before hook or an after hook. In these situations, we recommend using a before hook for the following reasons:

- If you forget to add the before hook to a particular spec, the failure will happen in *that* example rather than a later one.
- When you run a single example to diagnose a failure, the records will stick around in the database so that you can investigate them.

around

around hooks are bit more complex than what we've seen so far: they sandwich your spec code inside your hook, so part of the hook runs before the example and part runs after. Here's the setup and teardown logic from the previous snippet, wrapped in an around hook instead:

07-structuring-code-examples/04/around_hooks_spec.rb

```
RSpec.describe MyApp::Configuration do
  around(:example) do |ex|
    original_env = ENV.to_hash
    ex.run
    ENV.replace(original_env)
  end
end
```

The behavior of these two snippets is the same; it's just a question of which reads better for your application.

Before we move on to the concept of *scope*, let's take a quick look at one of the finer points of writing hooks: where to put the code.

Config Hooks

So far in this chapter, we've put our before, after, and around hooks inside example groups. If you only need your hooks to run for one set of examples, then this approach works fine. But if you need your hooks to run for multiple groups, you're going to get tired of copying and pasting all that code into each group.